

Lab 1: Image Sharpening

Operating Systems

Ayush
CS24BT001
IIT Dharwad

August 6, 2026

Objective

Implement an image sharpening pipeline in C++ using unsharp masking, reading and writing PPM images with the given `libppm` functions, and measure how long the pipeline takes to run.

Approach

Unsharp masking works in three steps:

1. **Smoothen** – replace every pixel with the average of its 3×3 neighborhood. Pixels on the border just average over whatever neighbors they actually have.
2. **Find details** – subtract the smoothened image from the original to get the high-frequency detail. Since pixel values are unsigned, negative differences are clamped to 0 instead of subtracting directly.
3. **Sharpen** – add the detail back onto the original image, clamped at 255, to get the final sharpened output.

While writing this, I initially summed the neighborhood directly into a `uint8_t`, which overflows since 9 pixels can sum to more than 255 – fixed by accumulating in a wider integer type before dividing. I also ran into the unsigned subtraction issue mentioned above, since a plain `input - smoothened` wraps around whenever a pixel is darker than its neighborhood average, which happens all over the image and not just at edges.

Timing is measured around `read_ppm_file`, S1, S2 and S3 using `std::chrono::high_resolution_clock`, and logged (along with the image number) both to the console and to `timing.txt`.

Timing Results

Ran the program on 7 test images of increasing size using `make run-sharpen INPUT=n OUTPUT=no`. Results from `timing.txt`:

Analysis

The bigger the input image, the longer the program takes to run – makes sense since every extra pixel adds a fixed amount of work across the three stages.

Image	Time (microseconds)
1	25,357
2	105,165
3	165,486
4	310,888
5	410,679
6	891,836
7	2,426,501

Conclusion

The pipeline smooths, extracts detail, and sharpens PPM images correctly once the overflow and underflow bugs in the pixel arithmetic were fixed.